

GCD loop explanation

1. to find GCD do we need to run a loop on the minimum number? #include<bits/stdc++.h> using namespace std; int main() { int num1,num2,gcd; cout<<"Enter The Two Number: "; cin>>num1>>num2; while(num2!=0) // imagine num2 is min? { int temp = num1 % num2; num1 = num2; num2 = temp; } cout<<"Gcd is "<<num1<<endl; }.2



1. to find GCD do we need to run a loop on the minimum number?

```
#include<bits/stdc++.h>
using namespace std;

int main()
{
    int num1,num2,gcd;
    cout<<"Enter The Two Number: ";
    cin>>num1>>num2;
    while(num2!=0) // imagine num2 is min?
    {
        int temp = num1 % num2;
        num1 = num2;
        num2 = temp;
    }
    cout<<"Gcd is "<<num1<<endl;
}
```

Good question — and nice code! 🙌

Let's break it down:

✓ Short answer:

No, you don't need to run a loop on the *minimum number*.

The Euclidean algorithm (which your code uses) automatically finds the **GCD efficiently**, regardless of which number is smaller.

💡 Explanation:

In the **Euclidean algorithm**, you repeatedly replace:

```
ini

num1 = num2
num2 = num1 % num2
```

until `num2` becomes `0`.

At that point, `num1` holds the **GCD**.

The logic works no matter which number is bigger because:

- If `num1 > num2`, `num1 % num2` just gives the remainder.
- If `num2 > num1`, then `num1 % num2` simply becomes `num1` (since `num1` is smaller), and the values swap automatically on the next step.

So there's **no need to loop from 1 to $\min(\text{num1}, \text{num2})$** or check which one is smaller — the algorithm handles it elegantly.



Example:

If `num1 = 15` and `num2 = 20`:

```
ini

temp = 15 % 20 = 15
num1 = 20
num2 = 15

temp = 20 % 15 = 5
num1 = 15
num2 = 5

temp = 15 % 5 = 0
num1 = 5
num2 = 0
```

→ GCD = 5 



Alternative (slower) method:

You *could* use a loop like this:

```
cpp

int gcd = 1;
for(int i = 1; i <= min(num1, num2); i++) {
    if(num1 % i == 0 && num2 % i == 0)
        gcd = i;
}
```

But this is **much slower** than the Euclidean method you wrote — especially for large numbers.

So your code is **already optimal** and correct 
You **don't** need to loop on the minimum number.